Typing the Case Studies of *Context Types* in Rocq: Extended Rules, Concrete Derivations, and Open Issues

Summary of the `ProofAutomation` / `CaseStudy` development

October 2026

Abstract

We report on mechanizing typing derivations for the case studies of the draft paper *Context Types: Bunched Modalities for Unifying Safety and Reachability*. The Rocq development already proves the Fundamental theorem for the core judgment; the case studies, however, only had basic (erased) typings. We added derivations in a case-study judgment $\Sigma; \Gamma \vdash_x e : \tau$, extended it with four rule families (a dependent variable rule, a capability case split, fixpoint unrolling, and paper-form constructor matches), and proved each new rule sound against the core denotation. We then derived the paper's examples, found several places where the paper's stated type is semantically false or vacuous, and isolated one remaining design issue in T-Fix. All structural steps of all derivations are checked (`Qed`); semantic side conditions are isolated as named obligations.

1 Setting and trust model

Judgment. `ProofAutomation/ExtendedTyping.v` defines $\Sigma; \Gamma \vdash_x e : \tau$ (`has_extended_context_type`). It contains every rule of the core judgment `has_context_type` (`typing_to_extended_typing`), plus case-study conveniences. Types are $\{\nu{:}b \mid \phi\}$ (demonic/over), $[\nu{:}b \mid \phi]$ (angelic/under), $\tau_1 \sqcap \tau_2$, $\tau_1 \oplus \tau_2$, $x{:}\tau_x \to \tau$, $x{:}\tau_x \mathrel{-\!\!*} \tau$ and $\Box\tau$; contexts are built with entangled ",", independent "$*$" and sum "$\oplus$". We write $\mathsf{P}(e)$ for the precise type $\{\nu{:}b \mid \nu = e\} \sqcap [\nu{:}b \mid \nu = e]$ that the operator rules produce, and $\mathsf{Eq}(v)$ for the match-equation type $\{\nu{:}b \mid \nu = v\} \sqcap [\nu{:}b \mid \nu = v]$. Paper `int` is translated to Rocq `nat` (truncated subtraction).

Semantic side conditions. T-Sub needs $\Gamma \vdash \tau_1 <: \tau_2$ (`sub_type_under`: $[\![\Gamma]\!] \models [\![\tau_1]\!]e \Rightarrow [\![\tau_2]\!]e$ for all well-typed e), and T-CtxSub needs $\Gamma_1 \preceq_X \Gamma_2$ (`ctx_sub_under`: every $r \models [\![\Gamma_1]\!]$ has some $r' \models [\![\Gamma_2]\!]$ with $r|_X \sqsubseteq r'$).

Trust model. Every derivation step is checked by Rocq (`Qed`). Each semantic side condition is a separately named lemma `<program>_obl_<what>`. Where an obligation is not machine-proved it is `Admitted` together with an informal argument against the Rocq denotation. Every case-study file was audited three times by adversarial reviewers who tried to build counterexample capabilities; none of the remaining obligations was refuted. The infrastructure (`ExtendedTyping.v`, `ExtendedSubsumption.v`, `ExtendedRuleSoundness.v`, `CaseTypes.v`) contains no `Admitted`. The project builds with `make`, and no file declares an `Axiom`.

File	Qed	Admitted	File	Qed	Admitted
`Section1Intro.v`	56	16	`MayMust.v`	84	23
`Section2Typing.v`	138	32	`BSTGenerator.v`	116	24
`Section3Logic.v`	50	9	`DisproveMerge.v`	96	13
`Section4Extension.v`	79	12	`ExtendedRuleSoundness.v`	189	0

2 Corrections to the case-study infrastructure

Angelic top. The aliases $U[b]$ used to be `under_ty b qual_top`. Because `qual_top` mentions no logic variable, its exact atom ranges over the empty set and $[\nu{:}b \mid \top]$ became the *trivial* under type, inhabited by every total term (proved: $\{\nu{:}b \mid q\} <: U[b]$ for any q). The paper's $[\nu{:}b \mid \top]$ means "every value of b is reachable". The

$$\frac{\Sigma \vdash^{\mathsf{WF}} \Gamma\ ,\ x{:}\tau \qquad \mathsf{fv}(\tau) \subseteq \mathsf{dom}\lfloor\Gamma\rfloor}{\Sigma;\ \Gamma\ ,\ x{:}\tau \vdash_x x : \tau}\ \text{T-VarComma}$$

$$\frac{\Sigma;\Gamma_a \vdash_x e : \tau \qquad \Sigma;\Gamma_b \vdash_x e : \tau \qquad \Gamma \preceq^{\vee}_{\mathsf{fv}(e)\cup\mathsf{fv}(\tau)} \Gamma_a \mid \Gamma_b}{\Sigma;\Gamma \vdash_x e : \tau}\ \text{T-CtxCase}$$

$$\frac{\mathsf{lc}(e_b) \qquad \Sigma;\Gamma \vdash_x \lambda a{:}s.\ e_b[f \mapsto \mathbf{fix}\ f.\lambda a.e_b] : \tau}{\Sigma;\Gamma \vdash_x \mathbf{fix}\ f.\lambda a{:}s.\ e_b : \tau}\ \text{T-FixUnroll}$$

$$\frac{\Sigma;\Gamma \vdash_x h :: t : \mathsf{Eq}(x) \qquad \Sigma;\Gamma \vdash_x e_c : \tau}{\Sigma;\Gamma \vdash_x \mathbf{match}\ x\ \mathbf{with}\ [\,] \to e_n \mid h :: t \to e_c : \tau}\ \text{T-ListMatchCons}$$

$$\frac{\Sigma;\Gamma_n \vdash_x [\,] : \mathsf{Eq}(x) \qquad \Sigma;\Gamma_c \vdash_x h :: t : \mathsf{Eq}(x) \qquad \Sigma;\Gamma_n \vdash_x e_n : \tau_n \qquad \Sigma;\Gamma_c \vdash_x e_c : \tau_c}{\Sigma;\Gamma_n \oplus \Gamma_c \vdash_x \mathbf{match}\ x\ \mathbf{with}\ [\,] \to e_n \mid h :: t \to e_c : \tau_n \oplus \tau_c}\ \text{T-ListMatchBoth}$$

$$\Gamma \preceq^{\vee}_X \Gamma_a \mid \Gamma_b \ \triangleq\ \forall r \models [\![\Gamma]\!].\ \exists r'.\ r|_X \sqsubseteq r' \wedge \big(r' \models [\![\Gamma_a]\!] \vee r' \models [\![\Gamma_b]\!]\big)$$

Figure 1: Rules added to the case-study judgment. The side condition of T-CtxCase also requires well-formedness and erasure agreement on X. The tree-match rules are analogous to the list ones, with $[\,] \mapsto \mathsf{Leaf}$, the pattern $h :: t$ replaced by $\mathsf{Node}(r, l, rr)$, and the extra premise $l \neq rr$. The Rocq rules use named binders (`lam_named`, `fixfun_named`, `match_list_named`).

aliases now use `q_base_all` $b \triangleq \nu \in b$, a qualifier that mentions ν. The demonic top $\{\nu{:}b \mid \top\}$ stays trivial, as intended. The same subtlety exists on paper: under Def. 4.7, $\mathsf{Atom}(\top)$ has $\mathsf{FV} = \emptyset$.

Generic subtyping lemmas. `ExtendedSubsumption.v` now proves transitivity, the $\sqcap$-projections and commutation, over-closure by qualifier entailment, and vacuous subtyping from an unsatisfiable context, all with `Qed`. Three previously admitted lemmas were *false* and were replaced by corrected, proved versions:

- $[\nu{:}b \mid q_1] <: [\nu{:}b \mid q_2]$ from $q_2 \Rightarrow q_1$ fails if q_2 is unsatisfiable, because $\mathsf{FUnder}(\mathsf{FAtom}\,\bot)$ has no model when worlds are non-empty. It also fails if q_2 mentions ν but q_1 does not. The corrected lemma adds $\mathsf{vars}(q_2) \subseteq \mathsf{vars}(q_1)$ and $\exists s.\, q_2(s)$.
- $\{\nu{:}b \mid q_o\} \sqcap [\nu{:}b \mid q_u] <: [\nu{:}b \mid q_o \wedge q_u]$ fails for $q_u = \top$. The corrected lemma adds $\mathsf{vars}(q_o) \subseteq \mathsf{vars}(q_u)$.

New lemmas proved along the way include $\tau_x \to \tau \ <:\ \tau_x \twoheadrightarrow \tau$ in every context, and closure facts for $\oplus$: for example $\{\nu{:}b \mid q_1\} \oplus [\nu{:}b \mid q] <: [\nu{:}b \mid q]$.

3 New rules

Figure 1 lists the rules added to $\vdash_x$. Each is justified by a theorem in `ExtendedRuleSoundness.v` of the same shape as the core Fundamental theorem: if the premises are semantically valid ($[\![\Gamma_i]\!] \models [\![\tau_i]\!]e_i$) and the side conditions hold, then $[\![\Gamma]\!] \models [\![\tau]\!]e$.

T-VarComma (`EXT_VarComma`; sound: `ext_var_comma_sound`). Core T-Var types x only in the singleton context $x{:}\tau$. The paper checks such a context under the basic context $\Sigma, \lfloor\Gamma\rfloor$ (WfComma), so τ may mention Γ. In Rocq, Σ is a fixed parameter of the judgment and type well-formedness is checked against $\lfloor\Gamma\rfloor$ only, so T-CtxSub alone cannot project $\Gamma\ ,\ x{:}\tau$ down to $x{:}\tau$ when τ depends on Γ. T-VarComma keeps the dependencies in the context; T-CtxSub does all other structural work (weakening, exchange, framing). It is what makes recursive calls such as $f\,(x-1)$, whose argument needs $\nu \prec x$, and variables with dependent types such as $y{:}\{\nu{:}\mathsf{nat} \mid \nu = x+1\}$ usable at all.

T-CtxCase (`EXT_CtxCase`; sound: `ext_ctx_case_sound`). This is a disjunctive form of T-CtxSub: every capability of Γ, projected on the free variables, is a capability of Γ_a or of Γ_b. It resolves the main expressiveness

gap we found. The match rules require either a split $\Gamma_t \oplus \Gamma_f$ into two *non-empty* summands (both branches reachable) or a uniform guard (one branch only). But a demonic binding $x{:}\{\nu{:}\mathsf{nat} \mid \top\}$ admits capabilities that reach only one branch and capabilities that reach both, so no single choice works. For instance, $x{:}\{\nu{:}\mathsf{nat} \mid \top\} \vdash$ **if** $x{=}0$ **then** 3 **else** 5 : $\{\nu{:}\mathsf{nat} \mid \nu{=}3 \vee \nu{=}5\}$ was not derivable before. With T-CtxCase the derivation splits the capabilities into "true only", "false only" and "both", and each case uses the existing match rules.

T-FixUnroll (`EXT_FixUnrollNamed`; sound: `ext_fix_unroll_named_sound_ext`, for *every* target type). A fixpoint may be typed by its one-step unfolding. This is the standard unrolling of incorrectness reasoning, used for the merge disproof (§4.4). The proof is a world-level value transport: two locally closed values with equal free variables whose applications are equivalent have the same type denotation (`ext_vt_main`). The premise $\mathsf{lc}(e_b)$ is necessary: a dangling bound variable would be captured by different binders in the fixpoint and in its unfolding.

Paper-form matches (sound: `ext_list_match_*_paper_rule_sound`, `ext_tree_match_*`). As in the paper's T-Match, the context of a reachable branch binds the pattern variables itself, with types chosen by the user. They are introduced beforehand by T-CtxSub, whose extension condition is the semantic content. The pre-existing cofinite rules forced the pattern binders to $\{\nu{:}\mathsf{nat} \mid \top\}$ and $\{\nu{:}\mathsf{list} \mid \top\}$, which makes the equation premise $h :: t : \mathsf{Eq}(x)$ unprovable.

An unsound first attempt. Our first version checked the branch premises under $\Gamma\ ,\ \Gamma_{pat}$ but concluded at Γ. That is unsound, because an unmodelable Γ_{pat} makes every premise vacuous. Rocq derives $\vdash_x$ **match** [1] $\ldots$: $\{\nu{:}\mathsf{nat} \mid \bot\}$ from that rule shape (`ext_list_match_cons_only_named_unsound`, kept as a machine-checked warning).

Named-binder variants. We also added T-LamD, T-LetD and T-Fix variants with concrete binders (`EXT_LamDNamedErased`, `EXT_LetDNamed`, `EXT_FixNamed`). They mirror the cofinite core rules in the same way as the pre-existing `EXT_LamNamed` and `EXT_LetNamed`; their soundness is not proved separately (§5).

4 Concrete derivations

4.1 Figure 6(a): may-analysis helps must-analysis

The program is $f = \lambda g.\lambda x.\lambda y.$ **let** $c = y > 10$ **in if** c **then** (**let** $gx = g\,x$ **in** $gx{+}1$) **else** $x{+}1$, with may-summary $\tau_g \triangleq \{\nu{:}\mathsf{nat} \mid \top\} \to \{\nu{:}\mathsf{nat} \mid \nu \neq 0\}$. We formalized the three settings proposed by the authors.

Setting (1) $x{:}[\nu{:}\mathsf{nat} \mid \nu = 0] \to y{:}[\nu{:}\mathsf{nat} \mid \nu \leq 10] \to [\nu{:}\mathsf{nat} \mid \nu = 1]$ is *semantically false*. Because "$\to$" entangles y with x, the capability

$$r = \{(x{=}0,\, y{=}11)\} \ \cup\ \{(x{=}1,\, y{=}n) \mid n \leq 10\}$$

satisfies both bindings: $x = 0$ is reachable, and every $y \leq 10$ is reachable, since y's qualifier does not mention x and is therefore not fibered by x. Yet f never returns 1: the store $(0, 11)$ takes the then-branch, where $g\,0{+}1 \neq 1$, and every other store returns 2. Matching on y is exactly what "loses" x's guarantee. No rule can derive this type, since the rules are sound.

Setting (2) $x{:}[\nu{:}\mathsf{nat} \mid \nu = 0] \to y{:}[\nu{:}\mathsf{nat} \mid \nu \leq 10] \twoheadrightarrow [\nu{:}\mathsf{nat} \mid \nu = 1]$ is valid and derived (`fig6a_2_typing`). After the three abstractions and the **let** of c, the body context is

$$\Gamma_2 = (g{:}\tau_g\ ,\ x{:}[\nu{:}\mathsf{nat} \mid \nu{=}0]) * y{:}[\nu{:}\mathsf{nat} \mid \nu{\leq}10]\ \ ,\ \ c{:}\mathsf{P}(y > 10).$$

The derivation splits on whether y's capability reaches a value > 10. The branch contexts are $\Gamma_T = g{:}\tau_g\ ,\ x{:}\{\nu{:}\mathsf{nat} \mid \top\}\ ,\ c{:}\mathsf{P}(\mathsf{true})$ and $\Gamma_F = g{:}\tau_g\ ,\ x{:}[\nu{:}\mathsf{nat} \mid \nu{=}0]\ ,\ c{:}\mathsf{P}(\mathsf{false})$.

$$\frac{\mathcal{D}_1 \qquad \mathcal{D}_2 \qquad \langle \Gamma_2 \preceq^{\vee} \Gamma_F \mid \Gamma_T \oplus \Gamma_F \rangle}{\Gamma_2 \vdash_x \textbf{if } c \textbf{ then } \ldots \textbf{ else } \ldots : [\nu{:}\mathsf{nat} \mid \nu{=}1]}\ \textsc{CtxCase} \qquad \mathcal{D}_1 = \frac{\Gamma_F \vdash_x c : \mathsf{P}(\mathsf{false}) \qquad \mathcal{D}_{else}}{\Gamma_F \vdash_x \textbf{if} \ldots : [\nu{:}\mathsf{nat} \mid \nu{=}1]}\ \textsc{MatchFalseOnly}$$

$$\mathcal{D}_2 = \dfrac{\dfrac{\Gamma_T \vdash_x c : \mathsf{P}(\mathsf{true}) \qquad \Gamma_F \vdash_x c : \mathsf{P}(\mathsf{false}) \qquad \mathcal{D}_{then} \qquad \mathcal{D}_{else}}{\Gamma_T \oplus \Gamma_F \vdash_x \textbf{if} \ldots : \{\nu{:}\mathsf{nat} \mid \nu{\neq}1\} \oplus [\nu{:}\mathsf{nat} \mid \nu{=}1]}\ \textsc{MatchBoth} \qquad \langle \oplus\text{-absorption} \rangle}{\Gamma_T \oplus \Gamma_F \vdash_x \textbf{if} \ldots : [\nu{:}\mathsf{nat} \mid \nu{=}1]}\ \textsc{Sub}$$

$$\mathcal{D}_{else} = \dfrac{\dfrac{}{\Gamma_F \vdash_x x+1 : \mathsf{P}(x+1)}\ \textsc{BinOp} \qquad \langle else_eq1_x0 \rangle}{\Gamma_F \vdash_x x+1 : [\nu{:}\mathsf{nat} \mid \nu{=}1]}\ \textsc{Sub}$$

$$\mathcal{D}_{then} = \dfrac{\dfrac{\ldots}{\Gamma_T \vdash_x g\,x : \{\nu{:}\mathsf{nat} \mid \nu{\neq}0\}}\ \textsc{AppFun} \qquad \dfrac{\Gamma'_T \vdash_x gx+1 : \mathsf{P}(gx+1) \qquad \langle then_ne1 \rangle}{\Gamma'_T \vdash_x gx+1 : \{\nu{:}\mathsf{nat} \mid \nu{\neq}1\}}\ \textsc{Sub}}{\Gamma_T \vdash_x \textbf{let}\ gx = g\,x\ \textbf{in}\ gx+1 : \{\nu{:}\mathsf{nat} \mid \nu{\neq}1\}}\ \textsc{Let}$$

where $\Gamma'_T = \Gamma_T\ , gx{:}\{\nu{:}\mathsf{nat} \mid \nu \neq 0\}$. The case-split obligation is where "$*$" matters. In the second case, every store of the g, x part is paired with some $y > 10$ (making c true) *and* with $y = 0$ (making c false). So $x = 0$ is still reachable in the $c = \mathsf{false}$ summand, which is what $\mathcal{D}_{else}$ needs. With "," this pairing fails: the counterexample of setting (1) is exactly such an unpaired capability. The may-summary of g enters only through *then_ne1*: the then-branch never produces 1, so it can be absorbed into the angelic $[\nu{:}\mathsf{nat} \mid \nu = 1]$.

Setting (3) $x{:}[\nu{:}\mathsf{nat} \mid \top] \to y{:}[\nu{:}\mathsf{nat} \mid (x{=}0 \Rightarrow \nu{=}12) \wedge (x{>}0 \Rightarrow \nu{\leq}10)] \to \tau_3$. The authors left the result type open. The strongest sound one is $\tau_3 = \{\nu{:}\mathsf{nat} \mid \nu \neq 1\} \oplus [\nu{:}\mathsf{nat} \mid \nu \geq 2]$, derived as `fig6a_3_typing`. Both branches are reachable: $x = 0$ forces $y = 12$, taking the then-branch, which never returns 1 by the may-summary of g. In the else-branch, every $x \geq 1$ is paired with $y = 0$, so $x+1$ reaches every $n \geq 2$. A result type mentioning $[\nu{:}\mathsf{nat} \mid \nu = 1]$ is unsound, because $x = 0$ is paired only with $y = 12$.

Figure 6(b). With the paper's summary $g : (\{\nu{:}\mathsf{nat} \mid \top\} \to \{\nu{:}\mathsf{nat} \mid \nu > 0\}) \sqcap ([\nu{:}\mathsf{nat} \mid \top] \to [\nu{:}\mathsf{nat} \mid \nu > 0])$, the stated type $x{:}\{\nu{:}\mathsf{nat} \mid \top\} \to \{\nu{:}\mathsf{nat} \mid \nu{=}2\} \oplus \{\nu{:}\mathsf{nat} \mid \nu{=}5\}$ is false. The function $g = \lambda n.n + 1$ satisfies both summaries, yet the singleton capability $x = 0$ yields only 2, and "$\oplus$" needs both parts non-empty. The angelic version $x{:}[\nu{:}\mathsf{nat} \mid \top] \to \ldots$ is derived. The safety-only type $x{:}\{\nu{:}\mathsf{nat} \mid \top\} \to \{\nu{:}\mathsf{nat} \mid \nu \geq 0\}$ is derived with two nested T-CtxCase splits on $a > 10$.

4.2 A demonic guard: the introduction's Peano subtraction

For $\lambda x.\lambda y.$ **let** $c = x \leq y$ **in if** c **then** 0 **else** $x{-}y$ at $\tau_a = x{:}\{\nu{:}\mathsf{nat} \mid \nu > 0\} \to y{:}\{\nu{:}\mathsf{nat} \mid \nu > 0\} \to \{\nu{:}\mathsf{nat} \mid \nu \leq x\}$ (`nat_minus_program_tau_a`), the guard is not determined by the demonic context $\Gamma_1 = x{:}\{\nu{:}\mathsf{nat} \mid \nu{>}0\}\ , y{:}\{\nu{:}\mathsf{nat} \mid \nu{>}0\}\ , c{:}\mathsf{P}(x \leq y)$. A context cannot directly say "the guard is not constant", so we nest two splits. The intermediate context $\Gamma_{\neg T} = \ldots\ , c{:}[\nu{:}\mathsf{bool} \mid \nu = \mathsf{false}]$ states "false is reachable". Its type is a closed under type, so no fibering is involved. Let Γ_T and Γ_F rebind c to $\mathsf{P}(\mathsf{true})$ and $\mathsf{P}(\mathsf{false})$, and let *if* abbreviate the conditional. Write $\tau_{\leq} = \{\nu{:}\mathsf{nat} \mid \nu \leq x\}$. The three branch derivations are

$$\mathcal{D}_T = \dfrac{\Gamma_T \vdash_x c : \mathsf{P}(\mathsf{true}) \qquad \dfrac{\Gamma_T \vdash_x 0 : \mathsf{P}(0) \qquad \langle zero \rangle}{\Gamma_T \vdash_x 0 : \tau_{\leq}}\ \textsc{Sub}}{\Gamma_T \vdash_x if : \tau_{\leq}}\ \textsc{MatchTrueOnly}$$

$$\mathcal{D}_F = \dfrac{\Gamma_F \vdash_x c : \mathsf{P}(\mathsf{false}) \qquad \dfrac{\Gamma_F \vdash_x x-y : \mathsf{P}(x-y) \qquad \langle false_result \rangle}{\Gamma_F \vdash_x x-y : \tau_{\leq}}\ \textsc{Sub}}{\Gamma_F \vdash_x if : \tau_{\leq}}\ \textsc{MatchFalseOnly}$$

$$\mathcal{D}_{TF} = \dfrac{\dfrac{\Gamma_T \vdash_x c : \mathsf{P}(\mathsf{true}) \qquad \Gamma_F \vdash_x c : \mathsf{P}(\mathsf{false}) \qquad \ldots}{\Gamma_T \oplus \Gamma_F \vdash_x if : \tau_{\leq} \oplus \tau_{\leq}}\ \textsc{MatchBoth} \qquad \langle sum \rangle}{\Gamma_T \oplus \Gamma_F \vdash_x if : \tau_{\leq}}\ \textsc{Sub}$$

and they are combined by two case splits:

$$\dfrac{\mathcal{D}_T \qquad \dfrac{\mathcal{D}_F \qquad \mathcal{D}_{TF} \qquad \langle case_rest \rangle}{\Gamma_{\neg T} \vdash_x if : \tau_{\leq}}\ \textsc{CtxCase} \qquad \langle case_true \rangle}{\Gamma_1 \vdash_x \textbf{if}\ c\ \textbf{then}\ 0\ \textbf{else}\ x - y : \tau_{\leq}}\ \textsc{CtxCase}$$

The guard premises ($\Gamma_T \vdash_x c : \mathsf{P}(\mathsf{true})$, etc.) are T-CtxSub followed by T-Var, and are proved. The same pattern derives the angelic corrected type $x{:}[\nu{:}\mathsf{nat} \mid \nu > 0] \to y{:}[\nu{:}\mathsf{nat} \mid \nu > 0] \mathrel{-\!\!*} [\nu{:}\mathsf{nat} \mid \nu > 0]$ for the paper's div guard (`div_guard_program_tau_b_wand`). There, the split is on whether $y = 0$ is reachable.

4.3 Persistence: two independent generator calls (§5)

For the generator form of e_{ND} (`e_ND_gen_program_extended_type`),

$$\textbf{let } f = \lambda_.\mathsf{gen}_{\mathsf{nat}}() \textbf{ in let } a = \mathsf{true} \textbf{ in let } y_1 = f\,a \textbf{ in let } y_2 = f\,a \textbf{ in } y_1 - y_2 \;:\; [\nu{:}\mathsf{nat} \mid \top],$$

the derivation is linear:

1. T-Let: $f : \square(\{\nu{:}\mathsf{bool} \mid \top\} \to [\nu{:}\mathsf{nat} \mid \top])$ by T-PersistIntro, and $a : \mathsf{P}(\mathsf{true})$. Let Γ_a be the resulting context.
2. T-Let: $\Gamma_a \vdash_x f\,a : [\nu{:}\mathsf{nat} \mid \top]$, using T-PersE as the subtyping $\square\tau <: \tau$ ⟨*persist_elim*⟩.
3. T-CtxSub: $\Gamma_a\ ,\ y_1{:}[\nu{:}\mathsf{nat} \mid \top] \preceq \Gamma_a * y_1{:}[\nu{:}\mathsf{nat} \mid \top]$ ⟨*comma_to_star*⟩. This holds because $\square$ makes the f-projection a singleton and a is precise, so the entangled context factors as a product.
4. T-LetD with $\Gamma_1 = \Gamma_a$: $\Gamma_a \vdash_x f\,a : [\nu{:}\mathsf{nat} \mid \top]$, and $y_1{:}[\nu{:}\mathsf{nat} \mid \top] * y_2{:}[\nu{:}\mathsf{nat} \mid \top] \vdash_x y_1 - y_2 : [\nu{:}\mathsf{nat} \mid \top]$ by BinOp and Sub ⟨*minus_sub*⟩.

The paper's duplication $x{:}\square\tau \preceq x{:}\square\tau * x{:}\square\tau$ is ill-formed in Rocq, where $*$ requires disjoint domains. Step 3 replaces it. With $f = \lambda x.z$ or $f = \lambda x.0 \oplus \lambda x.\mathsf{gen}() + 1$ instead of the generator, the corresponding judgments are semantically false, consistent with the paper's ✗ marks.

4.4 Disproving merge by unrolling (Fig. 8)

Let $\mathsf{O}[l] \triangleq \{\nu{:}\mathsf{list} \mid \nu = l\}$ and $\mathsf{P}[l] \triangleq \mathsf{P}(l)$. We derive (`merge_cex_extended_type`)

$$\vdash_x \mathit{merge} : \mathsf{P}[1;2] \mathrel{-\!\!*} \mathsf{P}[3;4] \mathrel{-\!\!*} \{\nu{:}\mathsf{list} \mid \neg \mathit{sorted}(\nu)\} \oplus \{\nu{:}\mathsf{list} \mid \top\},$$

the paper's disproof type instantiated at the counterexample inputs, since $\mathit{merge}\,[1;2]\,[3;4] = [1;3;2;4]$. The derivation has three unrolling levels; each one uses T-FixUnroll, so T-Fix is never used.

- **Level B** (`mu_fix00`): $\mathit{fix} : \mathsf{O}[\,] \to \mathsf{O}[\,] \to \mathsf{O}[\,]$. The steps are FixUnroll, Lam, Lam, then ListMatchNilOnly; the equation premise $[\,] : \mathsf{Eq}(xs)$ is Const followed by Sub ⟨*nil_eq*⟩, and the branch is T-VarComma.
- **Level A** (`mu_fix24`): $\mathit{fix} : \mathsf{O}[2] \to \mathsf{O}[4] \to \mathsf{O}[2;4]$.

$$\dfrac{\dfrac{\dfrac{\mathcal{E}_1 \qquad \mathcal{M}_{ys}}{G_{A1} \vdash_x \textbf{match } xs \;\ldots : \mathsf{O}[2;4]}\ \textsc{ListMatchCons} \qquad \langle pat_1 \rangle}{G_{A0} \vdash_x \textbf{match } xs \;\ldots : \mathsf{O}[2;4]}\ \textsc{CtxSub}}{\vdash_x \textbf{fix } \mathit{merge}.\ldots : \mathsf{O}[2] \to \mathsf{O}[4] \to \mathsf{O}[2;4]}\ \textsc{FixUnroll; Lam}^2$$

$$\mathcal{E}_1 = \dfrac{\dfrac{\ldots}{G_{A1} \vdash_x h_1 :: t_1 : \mathsf{P}(h_1 :: t_1)}\ \textsc{ListCons} \qquad \langle eq_1 \rangle}{G_{A1} \vdash_x h_1 :: t_1 : \mathsf{Eq}(xs)}\ \textsc{Sub}$$

$$\mathcal{M}_{ys} = \dfrac{\dfrac{\mathcal{E}_2\ \langle eq_2 \rangle \qquad \mathcal{D}_{step}}{G_{A2} \vdash_x \textbf{match } ys \;\ldots : \mathsf{O}[2;4]}\ \textsc{ListMatchCons} \qquad \langle pat_2 \rangle}{G_{A1} \vdash_x \textbf{match } ys \;\ldots : \mathsf{O}[2;4]}\ \textsc{CtxSub}$$

with $G_{A0} = xs{:}\mathsf{O}[2]\ ,\ ys{:}\mathsf{O}[4]$. Here $G_{A1} = G_{A0}\ ,\ h_1{:}\{\nu{:}\mathsf{nat} \mid \nu{=}2\}\ ,\ t_1{:}\mathsf{O}[\,]$ binds the pattern variables, introduced by the proved pattern lemma ⟨pat_1⟩. $\mathcal{D}_{step}$ continues as follows: a Let binds $t = \mathit{fix}\,t_1\,t_2 : \mathsf{O}[\,]$ by AppFun at level B, with the partial application typed by T-VarComma; a Let binds $lt = h_1{<}h_2 : \mathsf{P}(h_1 < h_2)$; then MatchTrueOnly ⟨*lt_true*⟩, a Let for $h_2 :: t$, and Sub on $h_1 :: (h_2 :: t)$ ⟨*leaf*⟩.

- **Level O**: the same shape at $\mathsf{P}[1;2] \to (\mathsf{P}[3;4] \mathrel{-\!\!*} \ldots)$, calling level A for $\mathit{merge}\,[2]\,[4]$. The final Sub to $\{\nu{:}\mathsf{list} \mid \neg \mathit{sorted}\} \oplus \{\nu{:}\mathsf{list} \mid \top\}$ is ⟨*unsorted*⟩: $[1;3;2;4]$ is unsorted. The outer "$\to$" becomes "$\mathrel{-\!\!*}$" through the general lemma $\tau_x \to \tau <: \tau_x \mathrel{-\!\!*} \tau$ ⟨*arrow_wand*⟩.

Every Admitted obligation here concerns a context in which all variables are constants, so each fibering has a single fiber.

4.5 Summary of all case studies

Paper item	Outcome in Rocq
§1 τ_a for div/minus	Derived; the minus variant uses nested CtxCase (§4.2).
§1 τ_b with only "$\to$"	*False*: on $\{x{=}n, y{=}m \mid 1 \le n \le m\}$, $x - y$ is always 0. The corrected "$-\!\!*$" version is derived. The informal p.2 fix (only f's parameter independent) is insufficient; both p.8 fixes are fully proved.
§2 contexts C_{dis}, C_{en}, Γ_{dis}, Γ_{en}	All ✓/✗ judgments are confirmed. $x - (z + y)$ is derived with T-LetI by giving $z + y$ a demonic type.
§2/§3 $\oplus$ examples, Ex. 3.4	Derived, including the demonic version $x{:}\{\nu{:}\mathsf{nat} \mid \top\} \vdash \mathbf{if}\ x{=}0 \ldots$ via CtxCase.
Ex. 3.1	The conclusion is derived, but the paper's intermediate premise $x{:}\{\nu{:}\mathsf{nat} \mid \nu < 10\}$, $y{:}\{\nu{:}\mathsf{nat} \mid \nu < 7\} \vdash x - y : \{\nu{:}\mathsf{nat} \mid \nu \le 3\}$ is *invalid* $((9, 0))$. The derivation must keep y precise.
Ex. 3.3 (sub)	True, but *not derivable*: the T-Fix issue of §5.
p.8 nested $f\,(f\,y)$	Right-nested A-normal form is not derivable, because **let** entangles r_2 with x. The monadic-normal-form nesting is derived.
§4 Ex. 4.5, 4.6	Derived; several fully proved.
§5 generators, $\Box$	$\mathsf{gen}_{\mathsf{bool}}() : \{\nu{:}\mathsf{bool} \mid \nu{=}\mathsf{true}\} \oplus \{\nu{:}\mathsf{bool} \mid \nu{=}\mathsf{false}\}$ derived; the generator form of e_{ND} derived; the non-persistent variants are false, as the paper marks.
Fig. 6(a)	(1) false; (2) and (3) derived (§4.1).
Fig. 6(b)	The demonic-x type is false; the angelic version and the safety version are derived.
Fig. 7 (bst_gen)	The stated type is *vacuously true*: $\Box(x{:}\{\nu{:}\mathsf{nat} \mid \top\} \to y{:}\{\nu{:}\mathsf{nat} \mid x < \nu\} \to [\nu{:}\mathsf{nat} \mid x < \nu < y])$ is uninhabited, because $x{=}1, y{=}2$ gives an empty exact angelic atom. The "$\le$" repair is false by totality. A precise-range version is true; its outer layers and range call are derived, and the rest is blocked by T-Fix.
Fig. 8 (merge)	The counterexample instance is derived by unrolling (§4.4). The literal type with $[\nu{:}\mathsf{list} \mid \top]$ parameters does not witness the bug: a correct merge also has it, via *merge* $[]\,l = l$. The version with sorted angelic parameters is blocked by T-Fix.

5 Open issues

1. The recursive binder of T-Fix is vacuous on mixed capabilities. T-Fix types the body under $f : (\{\nu{:}b \mid \phi\} \sqcap \{\nu{:}b \mid \nu \prec x\}) \to \tau$. The arrow denotation is a Kripke implication over whole-world argument extensions, with the argument type fibered by x. On any capability that contains a $\prec$-minimal x (for example $x = 0$, or $xs = []$) alongside larger values, no argument satisfies $\nu \prec x$ in the minimal fiber. The implication therefore holds vacuously, and *every* f satisfies the binding, including a diverging one. Since type denotations include totality, the premise of T-Fix is false whenever the body actually recurses, and no rule can derive it. This was found independently in Ex. 3.3, Fig. 7 and Fig. 8, and confirmed by audits, but it is not yet a machine-checked lemma. The paper's arrow denotation seems to share it. A plausible repair is to fiber the recursive binding by x, i.e. evaluate f's type per x-fiber ($x \triangleright \ldots$), as a new T-Fix variant with its own soundness proof. This decision is pending.

2. Unproved semantic obligations. 129 obligations remain `Admitted`. Most are of a few recurring kinds: $\oplus$-splits of contexts, operator-result subtyping on single or constant fibers, and $\Box$ elimination. They are not proved because the library lacks world-level introduction and elimination lemmas for FPlus, world filters, and precise types at $\mathbf{ret}\,x$. Adding these generically would discharge most of the obligations at once.

3. Soundness of the extended judgment as a whole. `ExtendedRuleSoundness.v` proves the *new* rules sound in isolation. The pre-existing convenience rules (`EXT_BinOp`, `EXT_AppOpValue`, `EXT_TreeNode`, `EXT_ListCons`, the named `Lam`/`Let`/`LamD`/`LetD`/`Fix` variants) still have no soundness proof. A full Fundamental theorem for $\vdash_x$, by induction reusing the per-rule lemmas, is the natural next step.

4. Smaller semantic mismatches with the paper.

- The angelic bottom $[\nu{:}b \mid \bot]$ is uninhabited under exact atoms, whereas the paper states $\{\nu{:}\mathsf{nat} \mid \nu > 0\} <: [\nu{:}\mathsf{nat} \mid \bot]$ (p.5).
- Persistent duplication needs the disjoint-domain workaround of §4.3.

- T-AppFun and T-AppOp take only variable arguments, so several programs were A-normalized; all such variants are documented in the files.

6 Conclusion

With four rule extensions, each proved sound against the core denotation, almost every case study of the paper now has a mechanized derivation whose structural part is checked by Rocq. Making the derivations explicit exposed several issues in the paper:

- a dependency bug in Fig. 6(a) setting (1), which the "$-\!\!*$" setting (2) repairs;
- a false type in Fig. 6(b);
- a vacuous type in Fig. 7;
- a non-witnessing disproof type in Fig. 8;
- an invalid intermediate judgment in Ex. 3.1;
- an unfaithful translation of $[\nu{:}b \mid \top]$.

The main remaining obstacle is the vacuity of T-Fix's recursive binder on mixed capabilities. Resolving it, for example with a fibered recursive binding, would unblock the three recursive examples: Ex. 3.3, Fig. 7 and the sorted-input form of Fig. 8.